For liquidity of an option say ESH7 we need to treat C and P differently for moneyness calculation done on minute level aggregated ES mid-prices. We get a value of moneyness for each strikes for example ESH7 C10000, ESH7 C10100, ESH7 C10200 .... Similarly Puts also We will have different Ranks of moneyness for each minute from ITM,ATM, OTM OTM (...-3, -2, 1) ATM (0) ITM (1,2,3...) Could you provide a strategy to aggregate the liquidty metrics (bid-ask spread and book depth) to a hour or day level?
Thought for a second

Yes. For option liquidity, you should **bucket/rank moneyness separately for calls and puts at the minute level**, then aggregate liquidity by those buckets to hourly/daily level. The key is: **do not aggregate across raw strikes directly**, because the same strike can move from OTM to ATM to ITM as the ES futures price moves.

A good framework is:

Minute-level option liquidity
→ classify each option by option-type-adjusted moneyness rank
→ aggregate across strikes within each moneyness bucket
→ aggregate across minutes into hour/day

1. Use option-type-adjusted moneyness

Let:

F_t = ES futures mid-price at minute t
K = option strike

For calls:

call_moneyness = F_t - K

For puts:

put_moneyness = K - F_t

Then the interpretation is consistent:

positive = ITM
zero-ish = ATM
negative = OTM

So instead of using `K - F_t` for both calls and puts, define:

adjusted_moneyness =
  F_t - K, if option_type = Call
  K - F_t, if option_type = Put

Then:

adjusted_moneyness > 0 → ITM
adjusted_moneyness ≈ 0 → ATM
adjusted_moneyness < 0 → OTM

2. Convert moneyness into ranks

Since ES options usually have fixed strike intervals, you can convert the distance from ATM into a discrete rank.

Let:

strike_interval = 100

Then:

moneyness_rank = round(adjusted_moneyness / strike_interval)

Example, if ES future mid-price is `9200`:

Instrument	Type	Strike	Adjusted moneyness	Rank	Bucket
ESH7 C9000	Call	9000	+200	+2	ITM
ESH7 C9200	Call	9200	0	0	ATM
ESH7 C9400	Call	9400	-200	-2	OTM
ESH7 P9000	Put	9000	-200	-2	OTM
ESH7 P9200	Put	9200	0	0	ATM
ESH7 P9400	Put	9400	+200	+2	ITM

This gives you a unified convention:

rank < 0 → OTM
rank = 0 → ATM
rank > 0 → ITM

Your example notation has `OTM (... -3, -2, 1)` — I assume that was meant to be `(... -3, -2, -1)`.

3. Minute-level liquidity fields

For every option instrument at every minute, calculate:

bid_ask_spread = ask_price - bid_price
relative_spread = (ask_price - bid_price) / option_mid_price
book_depth_bid = bid_size at best bid, or cumulative bid size over top N levels
book_depth_ask = ask_size at best ask, or cumulative ask size over top N levels
total_book_depth = book_depth_bid + book_depth_ask
imbalance = (book_depth_bid - book_depth_ask) / total_book_depth

For aggregation, I would strongly prefer using **relative spread** along with raw spread:

relative_spread = spread / option_mid_price

Raw spread is useful, but it is not directly comparable across deep ITM, ATM, and far OTM options because option prices differ greatly.

4. First aggregate within each minute and moneyness rank

At each minute, for each contract expiry, option type, and moneyness rank, aggregate all strikes that fall into that rank.

A useful grouping key is:

timestamp_minute
expiry
option_type
moneyness_rank

Then compute:

median_spread
weighted_avg_spread
median_relative_spread
weighted_avg_relative_spread
sum_book_depth
median_book_depth
num_quotes
num_valid_quotes

For spread, avoid simple averages if possible. Use either median or depth-weighted average.

Recommended spread aggregation

For each minute-rank bucket:

depth_weighted_spread =
  sum(spread_i * depth_i) / sum(depth_i)

where:

depth_i = bid_depth_i + ask_depth_i

This gives more influence to options with deeper, more meaningful quotes.

For robustness, also calculate:

median_spread
p25_spread
p75_spread

because option spreads can be noisy, especially for far OTM strikes.

Recommended book-depth aggregation

For depth, use:

sum_depth = sum(bid_depth_i + ask_depth_i)
median_depth = median(bid_depth_i + ask_depth_i)

Use `sum_depth` if you want the **total available liquidity in that moneyness bucket**.

Use `median_depth` if you want the **typical liquidity of an individual option at that moneyness rank**.

Both are useful and answer different questions.

5. Then aggregate minute buckets to hour/day

After you have minute-level values by moneyness rank, aggregate them to hourly or daily level.

Grouping key for hourly:

hour
expiry
option_type
moneyness_rank

Grouping key for daily:

date
expiry
option_type
moneyness_rank

Then compute time-weighted summaries:

avg_depth_weighted_spread
median_depth_weighted_spread
avg_relative_spread
median_relative_spread
avg_sum_depth
median_sum_depth
p10_depth
p90_depth
quote_availability

Where:

quote_availability = number of minutes with valid quotes / total trading minutes

This is very important. A far OTM option may show a good spread occasionally, but if it is quoted only 20% of the day, it is not truly liquid.

6. Recommended final hourly/daily liquidity panel

Your final table could look like this:

date/hour
expiry
option_type
moneyness_rank
moneyness_bucket
avg_spread
median_spread
avg_relative_spread
median_relative_spread
depth_weighted_spread
avg_total_depth
median_total_depth
sum_total_depth
quote_availability
num_options_observed

num_minutes_observed

Example:

Hour	Expiry	Type	Rank	Bucket	DW Spread	Median Rel Spread	Avg Depth	Quote Availability
10:00	ESH7	C	-3	OTM	1.25	8.1%	420	96%
10:00	ESH7	C	-2	OTM	0.90	5.4%	670	99%
10:00	ESH7	C	-1	OTM	0.65	2.8%	890	100%
10:00	ESH7	C	0	ATM	0.50	1.2%	1400	100%
10:00	ESH7	C	1	ITM	0.75	1.1%	760	98%

7. Do not mix calls and puts too early

I would keep calls and puts separate at least until the final reporting layer.

That means calculate separate liquidity curves:

Call liquidity by moneyness rank
Put liquidity by moneyness rank

Then, if you want a combined option-market liquidity metric, combine them later using weights.

For example:

```
combined_depth_weighted_spread =
  sum(spread_bucket * depth_bucket) / sum(depth_bucket)
```

across calls and puts.

But for analysis, calls and puts should remain separate because:

C rank -3 ≠ P rank -3 in terms of market demand, skew, and quote behavior

Both are OTM, but they often behave differently because downside puts are structurally more demanded in equity-index options.

8. Suggested aggregation hierarchy

I would use this hierarchy:

Level 1: Instrument-minute
ESH7 C10000 at 10:01
ESH7 C10100 at 10:01
ESH7 P10000 at 10:01

Level 2: Moneyness-rank-minute
Calls, rank -2, 10:01
Calls, rank -1, 10:01
Calls, rank 0, 10:01
Puts, rank -2, 10:01
Puts, rank -1, 10:01
Puts, rank 0, 10:01

Level 3: Moneyness-rank-hour/day
Calls, rank -2, hourly average
Puts, rank -2, hourly average

Level 4: Overall contract liquidity
ESH7 liquidity score for the hour/day

9. Build a contract-level liquidity score

Once you have hourly or daily liquidity by rank, you can create an overall liquidity score for ESH7.

For example, focus on the most relevant moneyness range:

ranks = [-5, -4, -3, -2, -1, 0, 1, 2, 3]

Then calculate:

```
liquidity_score =
  w1 * normalized_depth
 - w2 * normalized_spread
 - w3 * missing_quote_penalty
```

Or more simply:

liquidity_score = avg_depth / depth_weighted_relative_spread

A higher score means:

more depth + tighter spreads = better liquidity

I would not include extremely far OTM or deep ITM strikes in the main score unless they are important for your trading strategy. They can distort the result.

10. Practical recommendation

For your use case, I would produce three levels of output:

A. Liquidity surface

hour/date × option_type × moneyness_rank

This tells you where liquidity sits across the option chain.

B. OTM/ATM/ITM bucket summary

Group ranks into broader buckets:

Far OTM: rank <= -5
Near OTM: -4 to -1
ATM: 0
Near ITM: 1 to 4
Deep ITM: rank >= 5

Then aggregate liquidity for each bucket.

C. Contract-level liquidity score

One number for ESH7 per hour/day, based on selected ranks and weighted by depth or trading relevance.

My recommended approach

Use this pipeline:

1. Compute ES futures mid-price per minute.

2. For each option quote:
  adjusted_moneyness =
      F_t - K for calls
      K - F_t for puts

3. Convert adjusted moneyness to rank:

moneyness_rank = round(adjusted_moneyness / strike_interval)

4. Compute liquidity:
spread = ask - bid
relative_spread = spread / option_mid
depth = bid_depth + ask_depth

5. Aggregate instrument-minute data into:
minute × expiry × option_type × moneyness_rank

6. Aggregate minute-rank data into:
hour/day × expiry × option_type × moneyness_rank

7. Keep calls and puts separate.

8. Use:
depth-weighted spread for spread aggregation,
sum depth for total liquidity,
median depth for typical instrument liquidity,
quote availability for robustness.

The most important design choice is this: **rank moneyness at the minute level first, then aggregate.** Do not assign a fixed strike to a fixed moneyness bucket for the whole day, because ATM changes as ES moves.

what is strike_interval for an option contracts of ESH7?
Thought for a couple of seconds

For **ESH7 options** — options on the **March 2027 E-mini S&P 500 futures** — the strike interval is **not one fixed value across the whole chain**.

In practice, ES options commonly list strikes in **5-point, 10-point, 25-point, 50-point, or wider intervals**, depending on:

expiry type: weekly / monthly / quarterly
distance from ATM
current ES futures level
CME listing rules and active strikes

For your liquidity-ranking use case, I would **not hard-code `strike_interval = 100`**. That was only an example. CME/market data chains usually show much finer spacing near ATM. Secondary summaries commonly describe E-mini S&P futures options as having strike intervals of **5 or 10 index points**, while CME's live specs/option chains should be treated as the source of truth for the exact listed strikes at a given time.

Best practical approach

Infer the strike interval from the actual option chain each minute or day:

strikes = sorted(unique_strikes_for_ESH7)

strike_diffs = [strikes[i+1] - strikes[i] for i in range(len(strikes)-1)]

base_strike_interval = mode(strike_diffs)

But for moneyness ranking, I recommend something better:

atm_strike = strike closest to ES futures mid-price

rank = number of listed strike steps away from ATM

Instead of:

rank = round((adjusted_moneyness) / fixed_strike_interval)

Use the actual sorted strike ladder.

Example

Suppose at a minute:

ES mid = 9200
Listed strikes = [9100, 9150, 9200, 9250, 9300]
ATM strike = 9200

Then for calls:

C9100 → rank +2 ITM
C9150 → rank +1 ITM
C9200 → rank 0 ATM
C9250 → rank -1 OTM
C9300 → rank -2 OTM

For puts:

P9100 → rank -2 OTM
P9150 → rank -1 OTM
P9200 → rank 0 ATM
P9250 → rank +1 ITM
P9300 → rank +2 ITM

Recommendation

For ESH7 liquidity aggregation, use **actual strike-rank distance from ATM**, not a fixed strike interval.

That gives you stable buckets like:

rank -5, -4, -3, -2, -1, 0, +1, +2, +3, +4, +5

even if the chain has mixed intervals such as 5-point spacing near ATM and 25- or 50-point spacing farther away.